

2. Funciones

2.1. Parámetros y variables locales

Las funciones en JavaScript pueden ser definidas de manera tradicional mediante **function**. Como en la mayoría de los lenguajes de programación, las funciones admiten **parámetros** y permiten la declaración de **variables locales** en su interior. JavaScript permite también el acceso a variables definidas fuera de la función: son las **variables globales**. Esta característica es consecuencia del **ámbito léxico**, visto en la sección de **variables** del primer apartado de la unidad.

! IMPORTANTE

En general, es buena idea minimizar el uso de variables globales, ya que pueden crear problemas de mantenimiento del código.

Las variables globales son accesibles, a no ser que se defina una variable local con el mismo nombre, en cuyo caso la variable local es la que será referenciada (se dice que la variable local **enmascara** la variable global).

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A02_01.js**, con el código de ejemplo.

Los parámetros de las funciones pueden incorporar **valores por defecto**:

```
function pintar(color="negro") {  
    console.log(`Pintando con color ${color}`);  
}  
  
pintar(); // Pintando con color negro  
pintar("azul"); // Pintando con color azul
```

2.2. Funciones como valor

Las funciones en JavaScript son objetos, y, como tales, pueden ser almacenadas en variables:

```
function sumar(a, b) {  
    return a + b;  
}  
  
let suma1 = sumar(4,5); // suma1 = 9  
let funcSuma = sumar; // funcSuma almacena la función, no el resultado de su ejecución  
funcSuma(3,4); // 7  
let suma2 = funcSuma(3,6); // suma2 = 9
```

También pueden ser pasadas como parámetros a otras funciones:

```
function sumar(a, b) {  
    return a + b;  
}  
function restar(a, b) {  
    return a - b;  
}  
function operar(operacion, a, b) {  
    return operacion(a,b);  
}  
let v1 = operar(sumar, 4, 5); // v1 = 9  
let v2 = operar(restar, 4, 5); // v2 = -1
```

La aplicación más útil de esta característica es el uso de *callbacks*, que son funciones que se ejecutan cuando se completa una tarea.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U02_A02_02.js`, con el código de ejemplo.

En el ejemplo, el parámetro `si` de la función `confirmar` almacena la función que se pasa como parámetro; en este caso, `aceptar` [sin paréntesis]. Si el usuario responde afirmativamente en el cuadro de diálogo, se ejecuta dicha función mediante `si()` (como `si = aceptar`, `si()` equivaldrá a `aceptar()`). El uso de *callbacks* permite separar el código de la función principal de la acción que se ejecutará al completar la tarea.



IMPORTANTE

Es muy importante comprender cuándo hay que utilizar `()` y cuándo no al utilizar *callbacks*.

Estudiaremos los *callbacks* con más detalle en otra unidad.

2.3. Funciones anónimas

Las funciones anónimas son funciones que no tienen nombre definido. En otros lenguajes reciben el nombre de *funciones lambda*. Por ejemplo:

```
let sumar = function(a, b) {  
    return a + b;  
}  
  
let v1 = sumar(4, 5); // 9
```

En este caso, `sumar` es una variable que almacena una función anónima. También podríamos utilizar una función con nombre:

```
let sumar = function sumar(a, b) {  
    return a + b;  
}  
  
let v1 = sumar(4, 5); // 9
```

Ambos casos son muy parecidos. La principal diferencia radica en la **depuración del programa**: en el primer caso, la función no tendrá nombre; en el segundo caso, sí que aparecerá identificada con un nombre, por lo que su identificación será más sencilla.

Es muy común utilizar funciones anónimas en los *callbacks*. Así, el ejemplo anterior de la función **confirmar** quedaría de la siguiente manera:

```
function confirmar(pregunta, si, no) {
  if (confirm(pregunta))
    si();
  else
    no();
}

// En este caso, las funciones de callback se crean in situ como funciones
anónimas
confirmar("¿Sabes qué es un callback?",
  function() {
    console.log("Has respondido que sí");
  }, function() {
    console.log("Has respondido que no");
  });
```

! IMPORTANTE

Ten especial cuidado con el cierre de paréntesis y llaves al utilizar *callbacks* y funciones anónimas. Es muy común encontrarte con el patrón `})` como cierre.

2.4. Funciones flecha

Las funciones flecha permiten definir funciones mediante la siguiente sintaxis:

```
let func = (par1, par2, ... parN) => expresión;
```

Así, las siguientes funciones:

```
function sumar(a, b) {
  return a + b;
}

function restar(a, b) {
  console.log("Restando:");
  return a - b;
}
```

se podrían expresar como funciones flecha de la siguiente manera:

```
// El cuerpo de la función solo tiene una línea. Se pueden
// omitir las llaves y el 'return'
let sumar = (a, b) => a + b;

// El cuerpo de la función tiene varias líneas. Se deben
// incluir las llaves y el 'return'
let restar = (a, b) => {
  console.log("Restando");
  return a - b;
}
```

! IMPORTANTE

Estas funciones **no son simplemente versiones simplificadas** de las funciones convencionales, sino que tienen una serie de **características muy importantes** que hay que conocer. Algunas de ellas las comprenderemos más adelante, cuando tratemos los objetos. Dichas características son las siguientes:

- Son siempre **funciones anónimas**.
- No admiten los métodos **call**, **apply** y **bind**.
- No tienen su propio **this**, por lo que no son adecuadas para métodos de objetos.
- No pueden utilizarse como constructores con **new**.

2.5. Parámetros *rest* y sintaxis *spread*

En ocasiones, es interesante pasar a una función un **número variable de argumentos**. Por ejemplo, en una función que se encargue de calcular una media aritmética de un conjunto de números. En este caso, es útil el operador `...` delante del parámetro que vaya a recibir dichos datos, que pasarán a estar disponibles en un *array*.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A02_03.js**, con el código de ejemplo.

! IMPORTANTE

Si la función tiene muchos parámetros, solo uno de ellos, el **último**, puede utilizarse como parámetro *rest*.

Por otro lado, puede ser interesante realizar la **operación inversa**: queremos **convertir un *array* a un conjunto de parámetros**. Para ello se utiliza de nuevo el operador `...` delante del *array* que hay que convertir. Por ejemplo:

```
function sumar(a, b) {  
    return a + b;  
}  
  
let numeros1 = [4, 7];  
sumar(...numeros1); // 11
```

Uno de los usos más interesantes de este operador es la **copia de *arrays*** o la creación de *arrays* nuevos a partir de *arrays* existentes y elementos nuevos:

```
let colores1 = ["azul", "rojo", "verde"];  
let colores2 = colores1; // NO ES UNA COPIA, sino que hay un único array  
referenciado por dos variables  
let colores3 = [...colores1]; // colores3 almacena un ARRAY NUEVO, distinto  
del anterior  
let colores4 = ["cian", ...colores1, "rosa"]; // colores4 almacena un ARRAY  
NUEVO con nuevos elementos  
  
// Modifico el primer array  
colores2[0] = "amarillo";  
  
console.log(colores1[0]); // amarillo  
console.log(colores2[0]); // amarillo (mismo array)  
console.log(colores3[0]); // azul (array distinto)  
console.log(colores4.length); // 5  
console.log(colores4); // [ 'cian', 'azul', 'rojo', 'verde', 'rosa' ]
```


! IMPORTANTE

Estos conceptos tienen cierta complejidad, por lo que no se abordarán en profundidad. En las referencias puedes encontrar explicaciones más detalladas sobre el tema.

2.6. Ámbitos y *closures*

Tal como hemos adelantado en el primer apartado de la unidad, el **ámbito léxico** (*lexical scope*, en inglés) hace referencia a la **zona de influencia** de un determinado trozo de código. Así, podemos definir el **ámbito interno de una función** como el **cuerpo de esta**; definimos el **ámbito externo de una función** como el código que hay fuera de su cuerpo.

Una *closure* es una función que guarda referencias al ámbito exterior aun después de haberse ejecutado el código de dicho ámbito. Es muy habitual encontrarlas en **funciones que devuelven funciones**. Consideremos el siguiente caso clásico:

```
function crearContador() {  
  let cuenta = 0;  
  
  // Función closure  
  return function() {  
    // Referencia a la variable "cuenta" del ámbito exterior a la función  
    return cuenta++;  
  };  
}  
  
let contador1 = crearContador();  
console.log( contador1() ); // 0  
console.log( contador1() ); // 1  
console.log( contador1() ); // 2  
  
let contador2 = crearContador();  
console.log( contador2() ); // 0  
console.log( contador2() ); // 1
```

En dicho ejemplo podemos ver una función, **crearContador**, que devuelve una función cuando se ejecuta. Por tanto, las variables **contador1** y **contador2** hacen referencia a su vez a funciones, las cuales son *closures*, ya que guardan una referencia a la variable **cuenta**. Es importante ver que tanto **contador1** como **contador2** guardan una referencia a su propia versión de **cuenta**.

Lo difícil de entender de las *closures* es que el estado exterior no se elimina tras su ejecución, sino que sigue referenciado y por tanto sigue estando presente. En nuestro ejemplo, la variable **cuenta** parece que debería desaparecer una vez acaba la ejecución de **crearContador**: sin embargo, como dicha variable sigue referenciada por la función *closure*, sigue existiendo.

! IMPORTANTE

Las *closures* tienen mucha importancia en la **programación asíncrona**, ya que en ella se utilizan funciones que no se ejecutan inmediatamente, sino después de completar un proceso (petición de red, acceso a disco, tiempo de espera, etcétera).

También se utilizan como **alternativa a los objetos con propiedades privadas** (en el ejemplo anterior, la variable **cuenta** no es accesible desde el programa principal).